

**B.SC. THESIS DEFENSE · DEPARTMENT OF CSE · NPI UNIVERSITY OF BANGLADESH**

# UniBus Live

A Real-Time Bilingual University Bus Tracking System  
with Reliable Dual-Source GPS Acquisition

**Md. Shishir Hossain**

ID 0852220005101005 · Batch 12th

**Md. Sajib Hasan**

ID 0852220005101008 · Batch 12th

**Md. Showkat Hossen**

ID 0852220005101009 · Batch 12th

Supervisor: Mahbubur Rahman Chowdhury, Senior Lecturer, Dept. of CSE, NPIUB

**18 July 2026**

## THE PROBLEM

# Waiting for a bus you cannot see

A student at the stop has no way to know: did the bus leave 5 minutes ago — or is it 15 minutes away?



### Missed buses

The bus passed early; the student finds out too late.



### Long, uncertain waits

Standing in heat or rain for time that could be saved.



### No usable product

Commercial trackers: costly, operator-facing, English-only.

**Our goal:** let every student see their bus live, with the arrival time at their own stop — in their own language.

# UniBus Live — one platform, four parts

## PRIMARY SOURCE

### GPS Tracker

Concox GT06S fixed in the bus.

Fully automatic — no driver needed.

## FOR STUDENTS

### Passenger App

Live map · per-stop arrival times ·

alerts · English + বাংলা

## BACKUP SOURCE

### Driver App

Opt-in backup publisher —

used only when needed.

## FOR TRANSPORT OFFICE

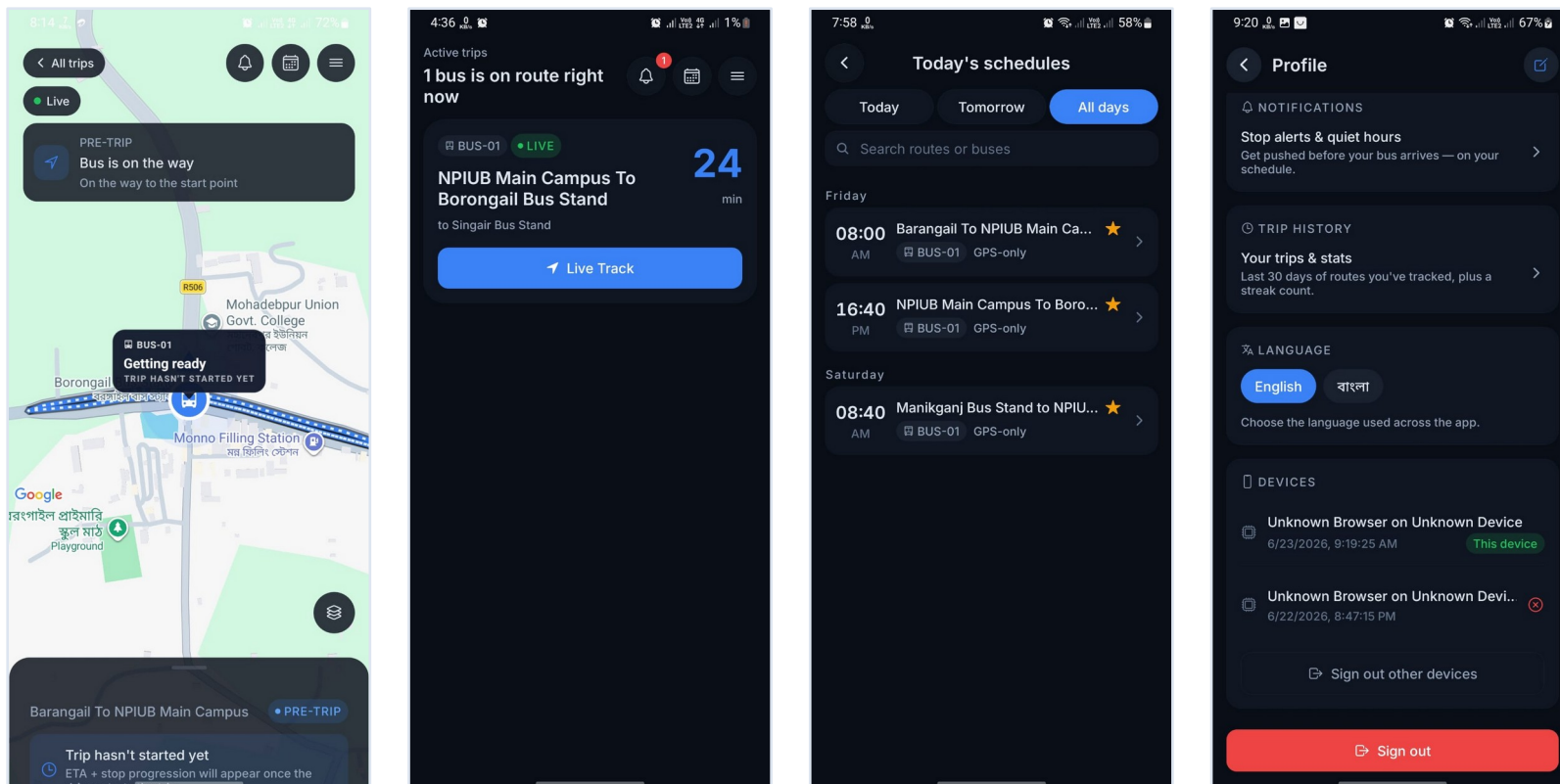
### Admin Console

Routes · schedules · fleet ·

live operations map.

Everything runs on **one low-cost server** — Node.js · PostgreSQL · Redis · Traccar · Socket.IO. No paid cloud services.

# The map is never blank



## Before departure

Bus visible at the depot — "getting ready", not a blank map.

## On route, live

Next stop and minutes away, updated in seconds.

## Today's schedules

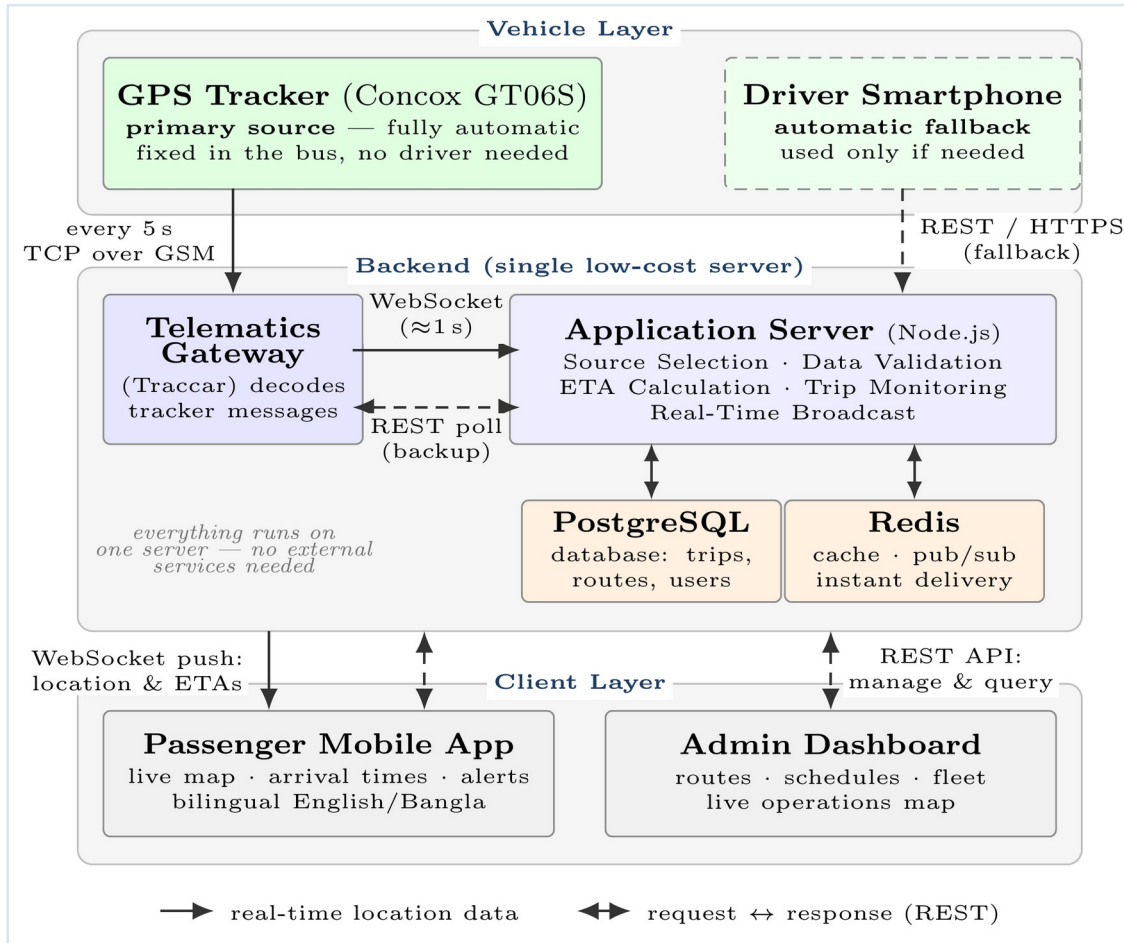
Every route and departure at a glance.

## English + বাংলা

The whole app in the rider's language.

Get-off alert: a push notification when the bus is within 80 m of your stop — even if the app is closed.

# Three layers, one low-cost server



## Vehicle layer

Tracker reports every 5 s over GSM; driver phone is only a fallback.

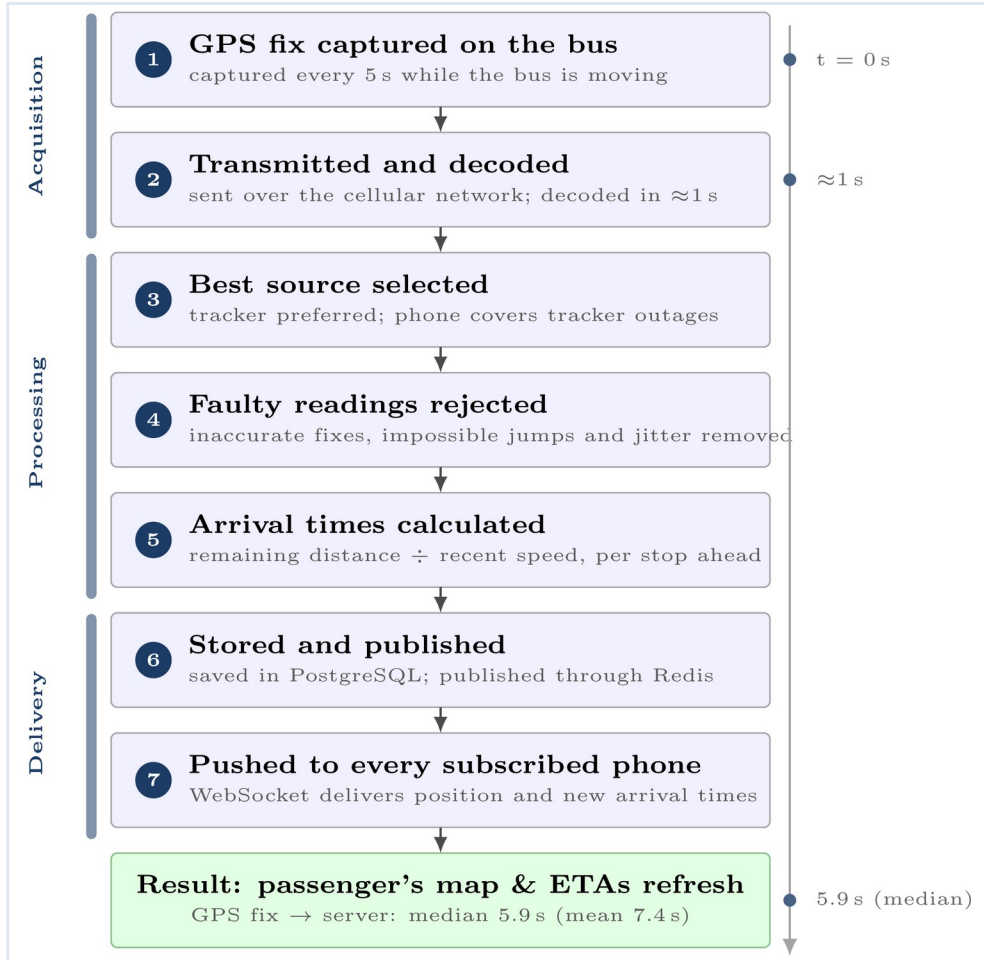
## One backend server

Traccar decodes · Node.js validates, computes ETAs, broadcasts · PostgreSQL + Redis.

## Client layer

Passenger app and admin console — pushed over WebSocket, not polled.

# From GPS fix to the student's screen



**5.9 s** median

from GPS fix on the bus to the server — measured over 46,286 real fixes.

**1-2** Capture & decode — every 5 s, decoded in  $\approx 1$  s

**3-4** Pick best source · reject faulty readings

**5** Compute the ETA for every stop ahead

**6-7** Store, publish, push to every subscribed phone

# Two independent sources, one automatic choice

## PRIMARY — GPS TRACKER

### Works completely on its own

- Hard-wired in the bus, reports every 5 s
- Trip starts automatically from its data
- No driver action, phone, or data plan needed

## BACKUP — DRIVER'S PHONE (OPT-IN)

### Only when a trip needs it

- Driver must be assigned and start sharing
- Covers buses without a tracker yet
- Normal operation never depends on it

**The server scores both sources on every fix and picks the best one automatically**

tracker preferred when healthy · switches mid-trip with no visible jump · what is automatic is the selection

# A simple estimator that works from day one

**ETA = remaining distance along the route ÷ recent average speed**

computed for every stop ahead, on every accepted GPS fix

## Route-aware

The fix is projected onto the route line, so sideways GPS error doesn't break the estimate.

## Honest confidence

Every estimate is labeled High / Medium / Low from the quality of the speed signal.

## No training data

Works from the first day — while logging every prediction so an ML model can come later.

Arrival radius 80 m — reaching it marks the stop and fires the rider's get-off alert.



# Seven weeks on the real fleet — zero simulation

**7**

weeks live  
6 May – 23 Jun 2026

**9**

routes

**3**

buses (1 with  
hardware tracker)

**38**

registered riders

**71**

completed trips

**46,548**

GPS reports ingested

Every number in this defense comes from the production database of this pilot.

# The map reflects reality within seconds

# 5.9 s

median latency, GPS fix → server

Bounded mainly by the 8 s polling interval — push delivery itself takes  $\approx 1$  s. A further 1.2 s (median) to persist each fix.

## Latency across 46,286 fixes (seconds)

Median



Mean



90th percentile



95th percentile



# The backup carried real weight

Which source tracked the 70 sourced trips?

**64% GPS tracker (45 trips)**

**36% phone backup (25)**

The 36% were mostly the two buses not yet fitted with trackers — coverage never stopped. Fleet-wide trackers will shrink this to a rare safety net.

**80%**

of arrival & service notifications  
were read by riders (36 of 45)

**80 m**

arrival radius that triggers  
the get-off alert

**0**

passenger-visible jumps during  
mid-trip source handovers

● A HIDDEN PROBLEM — FOUND & FIXED

# The pilot's most valuable discovery

98%

**of 46,548 GPS reports were pure waste**

buses kept transmitting every 5 seconds even while parked overnight — only 864 reports fell inside a real trip



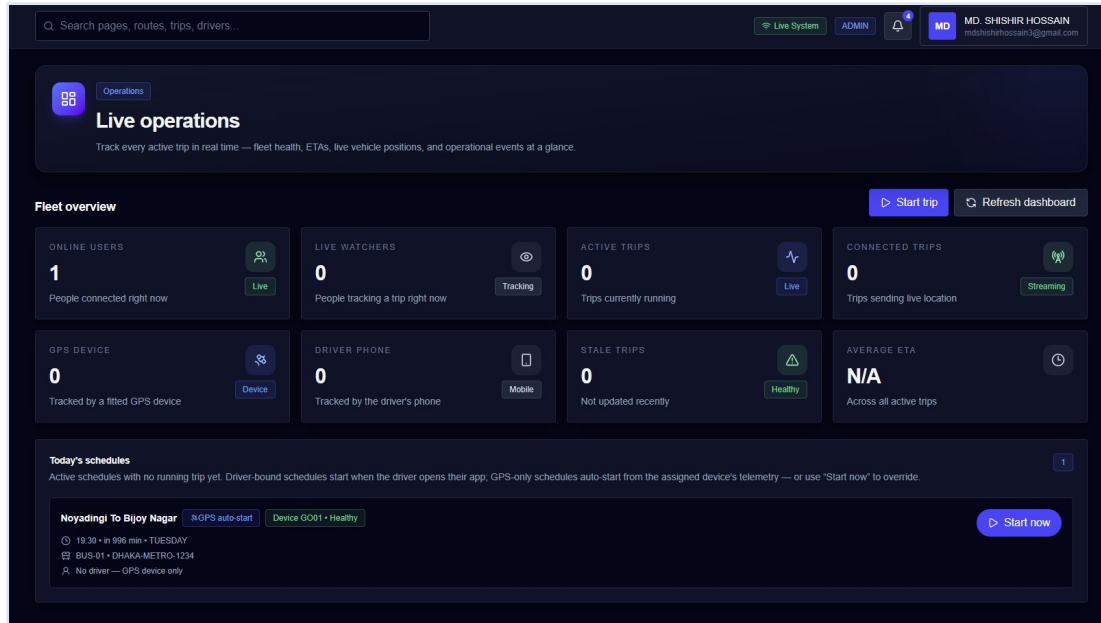
60x

**less parked-time reporting after our fix**

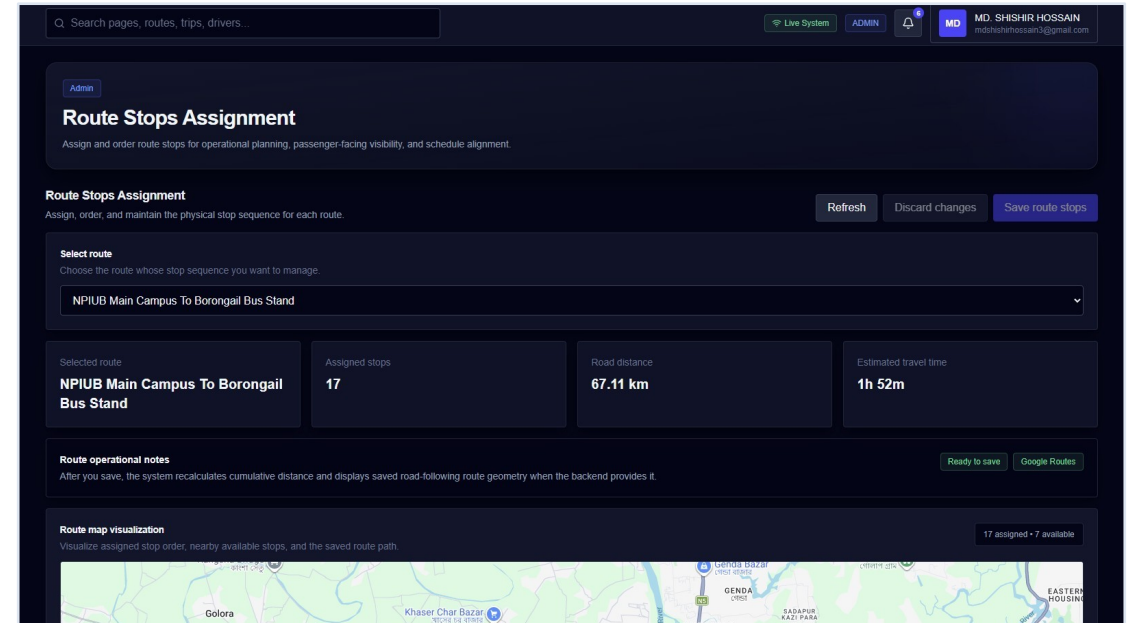
device: 5 s → 300 s when parked (one SMS: TIMER,5,300#) · server polling: 8 s → 60 s when idle

Invisible in design, unmistakable in real data — and fixed at both the device and the server **without touching live tracking.**

# The transport office sees everything



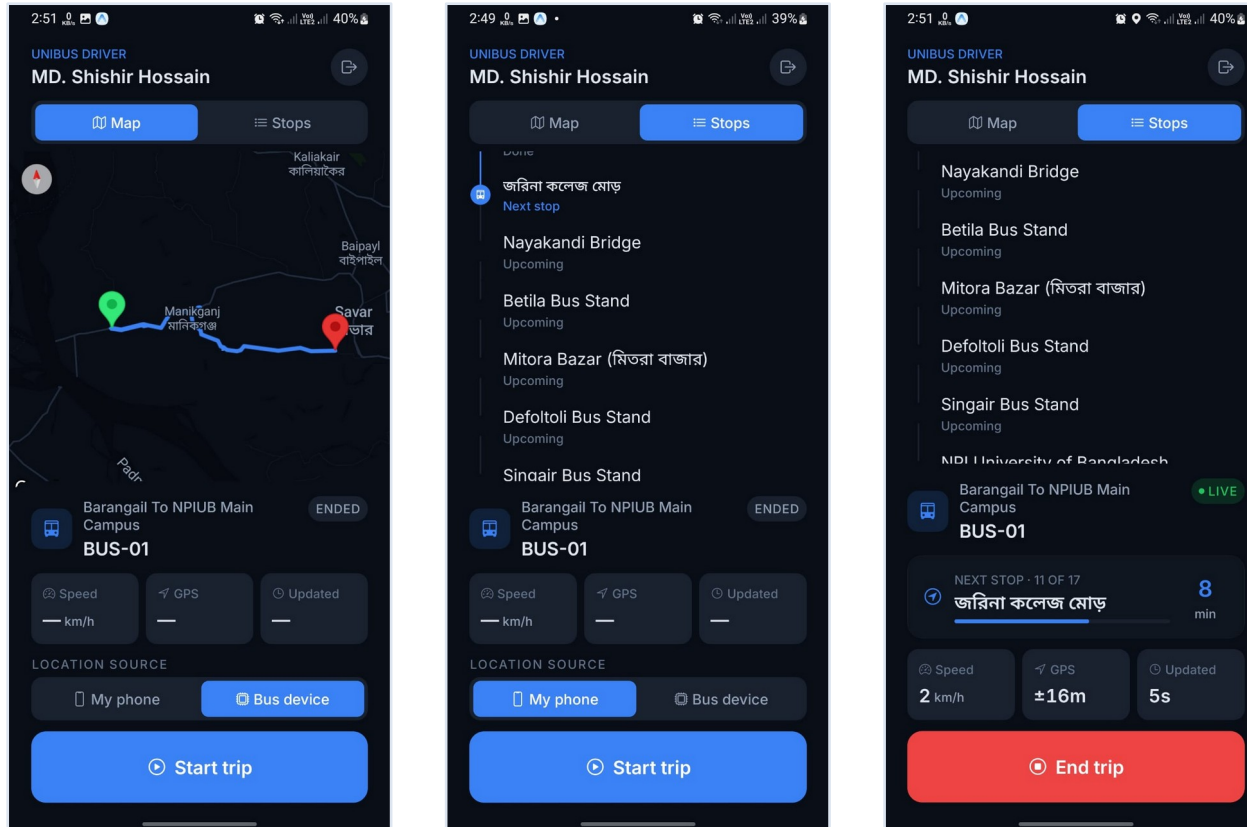
Live operations — whole fleet at a glance



Route builder — stops, distance, travel time on the map

Also: buses, GPS devices, schedules, stops, users, analytics — one web console, same API as the apps.

# The opt-in backup, kept deliberately simple



## Shares nothing by default

Location goes out only between Start trip and End trip.

## Choose the source

Bus device or my phone — visible on screen.

## Live self-check

Driver sees speed, GPS accuracy and update age while publishing.

## Drivers only

Assigned driver + driver role required — an authorization boundary.

# A feasibility pilot, and a roadmap

## LIMITATIONS

- Small pilot: 7 weeks, 3 buses, 38 riders
- Only one bus carried the hardware tracker
- ETA error (MAE) still accumulating — every prediction is logged for it
- Timetable linkage incomplete during the pilot

## FUTURE WORK

- ML arrival prediction trained on our logged trip history
- Trackers on every bus — phone becomes a rare safety net
- Ignition-wired (ACC) reporting: device sleeps with the engine
- Occupancy info and a formal usability study at larger scale

# Thank you

Questions welcome — every number is from our production database.

**5.9 s**

median latency

**46,548**

GPS reports

**71**

real trips

**64 / 36**

tracker vs phone

**60×**

idle waste cut

**80%**

alerts read